

EN UNA FRASE Sabes escribir tests automatizados del controlador con `@WebMvcTest` y `MockMvc`, simulando el servicio con `@MockBean`, sin arrancar un servidor real.

LO ESENCIAL

<code>spring-boot-starter-test</code>	Starter (alcance <code>test</code>) con JUnit 5, Mockito, MockMvc, AssertJ y Jackson. Ya viene en el <code>pom.xml</code> de Spring Initializr
Test unitario	Prueba una clase aislada, sin Spring (JUnit 5 + Mockito)
Test de integración de capa	Prueba el controlador con Spring MVC simulado (<code>@WebMvcTest</code> + <code>MockMvc</code>)
Test de extremo a extremo	Prueba la aplicación completa (<code>@SpringBootTest</code>); es el más lento
<code>@WebMvcTest(LibroController.class)</code>	Carga solo la capa web para ese controlador: ni servicios, ni repositorios, ni base de datos
<code>MockMvc</code>	Envía peticiones HTTP simuladas: <code>perform(get(...))</code> , <code>post</code> , <code>put</code> , <code>delete</code>
<code>@MockBean</code>	Registra un mock de Mockito como bean; el controlador lo recibe en lugar del servicio real
<code>ObjectMapper</code>	Convierte objetos Java en JSON para el cuerpo de la petición (<code>writeValueAsString</code>)
<code>when(...).thenReturn(...)</code> / <code>thenReturn(...)</code>	Configura qué devuelve o qué lanza el mock
<code>doThrow(...).when(mock).metodo(...)</code>	Lo mismo para métodos <code>void</code> , como <code>eliminar</code>
<code>any(Libro.class) / eq(1L)</code>	Matchers de Mockito: cualquier instancia del tipo / solo ese valor exacto
<code>verify(service).eliminar(1L) / verify(service, never())</code>	Comprueba que el mock se llamó, o que no se llamó
<code>andExpect(status().isOk())</code>	Verifica el código de estado (<code>isCreated</code> , <code>isBadRequest</code> , <code>isNotFound</code> , <code>isNoContent</code>)
<code>jsonPath("\$.title")</code> , <code>jsonPath("\$.length())</code>	Verifica campos del JSON: <code>\$</code> es la raíz, <code>\$.length()</code> el primer elemento del array

ERRORES TÍPICOS

- NoSuchBeanDefinitionException para BibliotecaService al arrancar el test**
 - ✓ `@WebMvcTest` no carga el servicio real; declara un `@MockBean` por cada dependencia del controlador.
- Body = null y No value at JSON path "\$.title"**
 - ✓ no has configurado el mock y Mockito devuelve `null` por defecto; pon el `when(...).thenReturn(...)` antes de la petición.
- Status 415, Content-Type 'application/octet-stream' not supported**
 - ✓ añade `.contentType(MediaType.APPLICATION_JSON)` en toda petición con cuerpo (POST y PUT).
- Los tests tardan segundos o fallan buscando la base de datos**
 - ✓ estás usando `@SpringBootTest`; para testear solo el controlador usa `@WebMvcTest` y reserva `@SpringBootTest` para el sistema completo.
- JSON path "\$.title" devuelve null en GET /libros**
 - ✓ la respuesta es un array; indexa primero: `$.length()`, y usa `$.length()` para el tamaño.

EN EL PROYECTO BIBLIOTECA

Se añade `LibroControllerTest` en `src/test/java/com/javaesencial/biblioteca/controlador`, con diez tests que cubren el CRUD: listado, búsqueda por id (200 y 404), creación (201 y 400), actualización (200 y 404) y borrado (204 y 404). El código de producción no cambia. Al ejecutar `mvn test` debes ver `Tests run: 10, Failures: 0, Errors: 0, Skipped: 0`.

■ CÓDIGO MÍNIMO Y PETICIONES

El esqueleto de la clase de test

(src/test/java/com/javaesencial/biblioteca/controlador):

```
@WebMvcTest(LibroController.class)
class LibroControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;

    @MockBean
    private BibliotecaService service;
}
```

JAVA

Un POST válido, con la convención debería_[resultado]_cuando[Condición]:

```
@Test
void debería_devolver201_cuandoLibroEsVálido() throws Exception {
    Libro entrada = new Libro(null, "Dune", "Frank Herbert", 1965);
    Libro creado = new Libro(4L, "Dune", "Frank Herbert", 1965);
    when(service.registrar(any(Libro.class))).thenReturn(creado);

    mockMvc.perform(post("/libros")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(entrada)))
        .andExpect(status().isCreated())
        .andExpect(jsonPath("$.id").value(4))
        .andExpect(jsonPath("$.titulo").value("Dune"));
}
```

JAVA

Un 404 cuando el servicio lanza la excepción:

```
@Test
void debería_devolver404_cuandoNoExisteElId() throws Exception {
    when(service.buscarPorIdOFallar(99L))
        .thenReturn(new LibroNoEncontradoException(99L));

    mockMvc.perform(get("/libros/99"))
        .andExpect(status().isNotFound());
}
```

JAVA

```
# Ejecutar todos los tests del proyecto
mvn test
```

TERMINAL

```
# Ejecutar solo la clase LibroControllerTest
mvn test -Dtest=LibroControllerTest
```